Real Kubernetes pod-event examples

Repository-backed mode: add the explicit GitHub overlay described in Owner-approved repository remediation to replace the direct-patch owner action below with a reviewed manifest PR, manual human merge and merge-commit CI gate. In that mode the button is **Approve & create PR**, and old proposals without repository evidence cannot be approved.

This opt-in integration targets Docker Desktop Kubernetes in the dedicated `remedi-demo` namespace. The original eight scenarios remain simulated. The four `k8s_*` scenarios read actual pod states, container terminations, Deployment/ReplicaSet ownership and Kubernetes events. They do not invent events.

Google ADK invokes read-only evidence/runbook tools through the existing Copilot bridge. A fresh model invocation reviews the proposal. Diagnosis is not a scenario-to-action lookup: neither an expected action nor a synthetic diagnostic story is sent to the model. Unsupported or ambiguous causes can escalate. Safety remains deterministic: AI cannot supply commands, patches, URLs or manifests.

Authorized boundaries

- Every real remediation requires an owner approval of the exact persisted proposal and field checkpoint, even when confidence and review agree.
- Only four explicitly named, labeled, single-container demo Deployments can be patched. Workloads outside this namespace are inaccessible to the service account.
- The account can get/patch those Deployments and get/list namespace pods, ReplicaSets and events. It cannot read Secrets, execute in pods, delete resources, create workloads, change RBAC or access other namespaces.
- The operator/admin creates deliberately failing examples separately. Pressing **Diagnose live pods** only requests diagnosis; it does not inject failures.
- Evidence sent to Copilot includes event messages, pod/container states and selected configuration fields. No pod logs, environment dumps, Secrets or kubeconfig are collected. Treat operational evidence as potentially sensitive and untrusted.
- This is a bounded local demo, not a general production Kubernetes auto-healer. Shared operator/owner role tokens do not provide independently attributed human identity.

Provision the examples

Docker Desktop Kubernetes must be running and `docker-desktop` must be reachable:

```
kubectl --context docker-desktop get nodes
.\scripts\setup-kubernetes.ps1 -Namespace remedi-demo -CreateExamples
```

Run from the project root. This creates the namespace if absent, the service account/Role/RoleBinding, and four Deployments. It refuses to reset existing managed examples. Review the manifests before running. No services or ingress are exposed. Each example requests 10m CPU and 16Mi memory, with a 100m CPU limit and an initial 32Mi memory limit. Images require Docker Hub access.

Scenario	Live failure	Bounded change proposed when evidence supports it
`k8s_crashloop`	Demo CRASH flag causes startup exit, BackOff/CrashLoopBackOff	Change only the demo `CRASH` value from `true` to `false`
`k8s_oom`	Allocate 64Mi under a 32Mi limit; actual last termination is OOMKilled	Change only the memory limit from 32Mi to 128Mi
`k8s_imagepull`	Known missing `python:remedi-missing-demo` tag produces ErrImagePull/ImagePullBackOff	Replace only that image with `python:3.12-alpine`
`k8s_pending`	Impossible `remedi.demo/unavailable=true` selector produces FailedScheduling	Remove only that selector key

The example images are public tags, not production supply-chain pins. The memory action changes a Deployment limit, not node capacity. OOMKilled is a container termination reason and may appear while the current pod status is CrashLoopBackOff. Do not diagnose until the relevant real evidence has appeared.

Credentials and Docker

Setup requests a short-lived service-account token (24 hours; the API server can shorten this) and writes it and the cluster CA into ignored `.runtime\kubernetes`. The application does **not** receive the administrator kubeconfig. Never print, commit, upload or place these files inside an image. Protect the directory with your Windows account's file permissions. The Docker build context excludes it.

Renew credentials without changing workloads:

```
.\scripts\setup-kubernetes.ps1 -Namespace remedi-demo
```

The adapter rereads the token file for each request, so renewal does not need an application restart for token-only rotation. If the cluster endpoint or TLS identity changes, recreate the backend with the overlay after refreshing setup. Expiry/401/403/TLS/network failures are surfaced explicitly; no simulator fallback or disabled TLS verification is used.

Enable the explicit Compose overlay:

```
docker compose -f compose.yaml -f compose.kubernetes.yaml up -d --build --wait
```

This builds local working-tree images with the default `latest` tag. It is not a SHA-certified Actions deployment. Existing named database storage is preserved. Use the tested CI/CD path for commit-tagged deployment. Docker containers reach the API at the endpoint discovered by setup in the selected `docker-desktop` context. Docker Desktop's newer clusters can use a dynamic loopback port rather than 6443. Setup maps loopback transport to `host.docker.internal` at that exact port and preserves the context's TLS server name (or original server hostname) for certificate identity verification, analogous to Kubernetes `tls-server-name`. The generated private-directory `connection.env` supplies both values. The CA and hostname are still verified; this does not skip certificate checks or expose a proxy. If the endpoint is unavailable, resolve Docker Desktop networking; do not disable TLS.

Walkthrough: diagnose first, approve separately

1. Open http://127.0.0.1:8082 and connect with the **operator** token.
2. Confirm the sidebar says **REAL KUBERNETES ENABLED** and `remedi-demo`.
3. Select **Kubernetes OOMKilled** and press **Diagnose live pods**.
4. Watch **Remediation progress** and the live audit timeline: evidence collection, model tool calls, observed evidence, proposal, fresh AI review, exact field checkpoint, and the policy decision.
5. Inspect **Evidence & telemetry** on the observation event. Look for actual OOMKilled, the 32Mi limit, pod UID and Kubernetes event reason/message.
6. Inspect **Exact Kubernetes change for approval**: namespace/Deployment, before/after values and rollback scope. Review the AI recommendation and judge event.
7. At `awaiting_approval`, no remediation patch has been applied. Leave it here when only diagnosis has been authorized.
8. To authorize the real change, reconnect with the **owner** token, reselect the incident, enter a meaningful reason and click **Approve change**.
9. Watch apply/rollout and the measured consecutive readiness samples. Closure requires a current observed Deployment generation, exactly one updated/available replica, an owned current-revision Ready pod and no container restarts, on three consecutive checks of the same pod identity.
10. Export the audit JSON and confirm actual cluster state independently:

```
kubectl --context docker-desktop -n remedi-demo get deployments
kubectl --context docker-desktop -n remedi-demo get pods
kubectl --context docker-desktop -n remedi-demo get events --sort-by=.metadata.creationTimestamp
```

Repeat for the other three examples. The workflow never silently approves them. One active incident per target prevents competing approval plans. To repeat an already repaired example, a human must deliberately reset that specific workload after resolving/rejecting its active incident; the application does not reinject faults.

What progress means

The UI shows recorded workflow stages, not artificial percentages or an ETA. Every rollout sample is persisted and streamed with actual pod state, replica counts and consecutive passes. Normal checks are two seconds apart, with a 120-second rollout deadline; configuration bounds are validated.

- `closed`: three consecutive real readiness checks passed.
- `rolled_back`: recovery timed out and the approved configuration field was restored and compared with its checkpoint. **The original failing workload is still unresolved**; rollback is not healthy recovery.
- `escalated` / `rejected`: no remediation was applied.
- `failed`: inspect both audit and cluster state. A network error or restart after patch submission may leave a change applied; no blind replay occurs.

An approval becomes stale if the Deployment UID or pod template changes. The application compares the approved checkpoint, then uses Kubernetes JSON Patch UID/resourceVersion tests for an atomic update. Status-only changes do not invalidate approval. A failed rollback never overwrites an externally changed template. A failed diagnostic/reviewer call does not execute a remedy.

CI/CD integration

Hosted CI remains cluster-free: mock-adapter/backend tests and mock-API UI tests validate real-mode gates and progress without live inference or writes. Existing simulator tests still run.

To retain Kubernetes mode on future Actions deployment, explicitly set repository Actions variable `REMEDI_KUBERNETES_CREDENTIALS_DIR` to this machine's absolute private `.runtime\kubernetes` directory. This is only a path, not token data. The runner must be able to read it; refresh the token locally as needed. The default namespace is `remedi-demo`. The workflow passes the path to `deploy.ps1`, which selects the overlay and verifies the owner-gated configuration. Leaving the variable unset deploys the original simulation-only configuration. No variable has to be set for ordinary local Compose use.

The existing optional `live_smoke` still runs only the synthetic cache scenario. It never approves or remediates Kubernetes workloads. Image rollback retains database storage; it does not undo real cluster changes already approved.

Troubleshooting

Symptom	Check
Kubernetes examples missing	Explicit overlay, `/api/config` Kubernetes enabled, current frontend image
401 / 403	Renew token, verify namespace Role/RoleBinding; do not give cluster-admin
TLS / connection error	Docker Desktop Kubernetes enabled, container DNS/API endpoint and saved CA
No failure evidence yet	Wait for container startup/backoff, image pull or scheduler events; inspect pods directly
Model escalates	Inspect actual evidence and model justification; do not bypass policy
Target changed / stale approval	Reject/resolve old incident and diagnose again; never reuse its checkpoint
Queue conflict	A prior incident for that target is still active; review it first
Timeout / rollback	Inspect pod events and resources; checkpoint restoration is not recovery
App restart after patch	Workflow fails without replay; inspect actual Deployment before further action